

**DESIGN AND PERFORMANCE ANALYSIS OF A
PIPELINED PROCESSOR WITH A MULTI-LEVEL
CACHE MEMORY SYSTEM**

CSA1221 – Computer Architecture

Report Based on the Assignment Required Deliverables

Analytical Processor and Memory Hierarchy Design

Edge-Computing Processor Design Study

BATCH :

1. Introduction

This report documents the analytical design of a single-core edge-computing processor and its supporting memory hierarchy. The design begins with register-transfer notation and a multiple-bus datapath for load, store, add and branch instructions. It is then extended into a five-stage pipeline and a two-way superscalar design with out-of-order and speculative execution.

The same report evaluates a three-level cache hierarchy, virtual memory support, SRAM/DRAM technology choices, Average Memory Access Time (AMAT), bandwidth and throughput. Each required deliverable is presented using the structure Purpose, Design/Pseudocode, Sample Input, Sample Output and Result / Observation, following the format of the sample deliverables report.

2. Deliverable Mapping

Deliverable	Implementation Area
1	Register-transfer notation and multiple-bus datapath design
2	Load, store, add and branch data movement analysis
3	Five-stage pipeline design: IF, ID, EX, MEM and WB
4	Data and control hazard identification with CPI comparison
5	2-way superscalar dispatch and execution design
6	Out-of-order, speculative execution and misprediction recovery
7	L1/L2/L3 cache hierarchy and virtual memory design
8	AMAT calculation and memory technology justification
9	Bandwidth and throughput comparison of design alternatives

10	Reflection, SDG 9 relevance, learning outcomes and viva readiness
----	---

Deliverable 1: Register-Transfer Notation and Multiple-Bus Datapath

Purpose

The purpose is to model the movement of operands and results among the register file, ALU and memory. A three-bus organization is selected: Bus A and Bus B carry two source operands simultaneously, while Bus C returns the ALU or memory result to the destination register.

Design / Pseudocode

```
START
FETCH instruction from instruction memory
DECODE opcode and select source/destination registers
```

```
Bus A <- RegisterFile[rs]
Bus B <- RegisterFile[rt]
```

```
IF instruction = ADD
    ALUout <- Bus A + Bus B
    RegisterFile[rd] <- ALUout
END IF
```

```
IF instruction = LOAD
    ALUout <- Bus A + sign_extended_offset
    MDR <- Memory[ALUout]
    RegisterFile[rt] <- MDR
END IF
```

```
IF instruction = STORE
    ALUout <- Bus A + sign_extended_offset
    Memory[ALUout] <- Bus B
END IF
```

```
IF instruction = BRANCH
    ALUout <- Bus A - Bus B
    IF ALUout = 0
        PC <- PC + 4 + branch_offset
    END IF
END IF
STOP
```

Sample Input

Instruction	Operands
ADD R3, R1, R2	R1=12, R2=8
LOAD R5, 16(R1)	R1=1000, Memory[1016]=45
STORE R4, 20(R1)	R1=1000, R4=90
BEQ R1, R2, LABEL	R1=R2

Sample Output

MULTIPLE-BUS DATAPATH EXECUTION

Bus A -> source operand 1

Bus B -> source operand 2

ALU -> arithmetic / address / comparison

Bus C -> result written back to register file

ADD : $R3 \leftarrow R1 + R2 = 20$

LOAD : $EA \leftarrow 1000 + 16 = 1016$; $R5 \leftarrow \text{Memory}[1016] = 45$

STORE : $EA \leftarrow 1000 + 20 = 1020$; $\text{Memory}[1020] \leftarrow 90$

BEQ : $R1 - R2 = 0$; branch target selected

Result / Observation

The three-bus organization allows two operands to be read concurrently and one result to be written back, reducing bus conflicts and providing an efficient basis for pipelining.

Deliverable 2: Instruction-Level Datapath Operation Analysis

Purpose

This deliverable explains how each instruction uses the datapath resources and identifies the register-transfer sequence required for correct execution.

Design / Pseudocode

FOR each instruction

FETCH: $IR \leftarrow \text{InstructionMemory}[PC]$; $PC \leftarrow PC + 4$

DECODE: read register operands

CASE opcode OF

LOAD : EX address -> MEM read -> WB register

STORE : EX address -> MEM write

ADD : EX ALU addition -> WB register

BEQ : EX compare -> update PC if condition true

END CASE

END FOR

Sample Input

Instruction sequence: LOAD R1, 0(R2) → ADD R3, R1, R4 → STORE R3, 8(R2) → BEQ R3, R0, EXIT

Sample Output

Instruction	Register Transfer / Main Resource
LOAD R1,0(R2)	EA=R2+0; MDR=Memory[EA]; R1=MDR
ADD R3,R1,R4	ALUout=R1+R4; R3=ALUout
STORE R3,8(R2)	EA=R2+8; Memory[EA]=R3
BEQ R3,R0,EXIT	ALU compares R3 and R0; PC updated if equal

Result / Observation

The instruction analysis confirms that load uses all five major processing functions, store does not require register write-back, ADD bypasses data memory, and branch primarily uses comparison and PC selection.

Deliverable 3: Five-Stage Pipeline Design

Purpose

The datapath is divided into IF, ID, EX, MEM and WB stages so that multiple instructions can be processed simultaneously at different stages.

Design / Pseudocode

PIPELINE STAGES

IF : Fetch instruction and increment PC

ID : Decode instruction and read registers

EX : Perform ALU operation / address calculation / branch comparison

MEM : Access data memory for load/store

WB : Write result into register file

Pipeline registers:

IF/ID -> ID/EX -> EX/MEM -> MEM/WB

Sample Input

Four instructions enter the pipeline in consecutive cycles.

Sample Output

Cycle	I1	I2	I3	I4
1	IF			
2	ID	IF		
3	EX	ID	IF	
4	MEM	EX	ID	IF
5	WB	MEM	EX	ID
6		WB	MEM	EX

7			WB	MEM
8				WB

Result / Observation

After the pipeline is filled, the ideal scalar design can complete approximately one instruction per cycle, subject to hazards and memory delays.

Deliverable 4: Hazard Analysis and CPI With/Without Mitigation

Purpose

This deliverable identifies at least two data hazards and one control hazard and compares pipeline performance before and after forwarding, stalling and branch prediction.

Design / Pseudocode

INSTRUCTION SEQUENCE

I1: LOAD R1, 0(R2)

I2: ADD R3, R1, R4 <- RAW hazard on R1

I3: ADD R5, R3, R6 <- RAW hazard on R3

I4: BEQ R5, R0, TARGET <- RAW + control hazard

WITHOUT mitigation:

insert conservative stalls for dependent values

resolve branch late and flush wrong-path instructions

WITH mitigation:

forward ALU results where possible

retain one load-use stall when required

use branch prediction and flush only on misprediction

Sample Input

Assumed workload: 100 instructions; 30% dependent operations; 20% branches; branch misprediction rate = 10%; branch penalty = 2 cycles.

Sample Output

CPI WITHOUT HAZARD MITIGATION

Base cycles = 100

Data-hazard stall cycles (estimate) = 45

Control-hazard penalty = 40

Total cycles = 185

CPI = $185 / 100 = 1.85$

CPI WITH FORWARDING + PREDICTION

Base cycles = 100

Remaining load-use stalls = 10
 Misprediction penalty = $20 * 0.10 * 2 = 4$
 Total cycles = 114
 CPI = $114 / 100 = 1.14$

Result / Observation

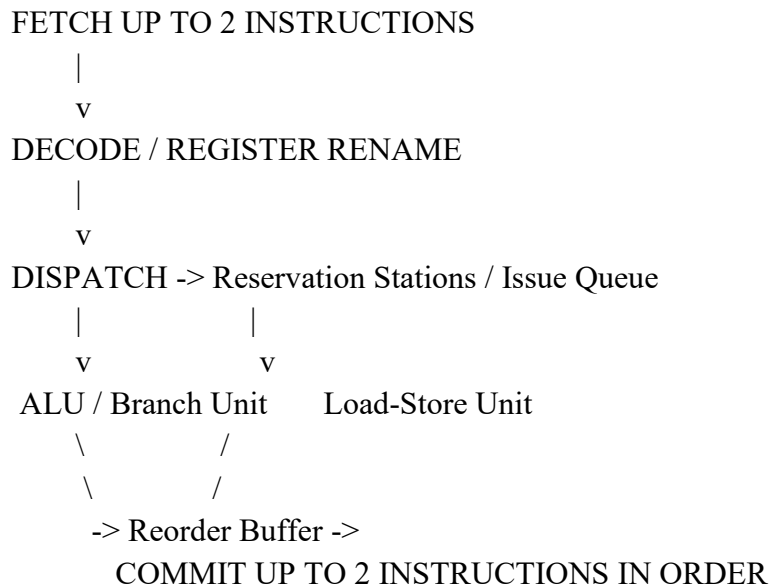
Hazard mitigation reduces CPI from approximately 1.85 to 1.14. Forwarding removes most register dependencies, while branch prediction significantly reduces the average control-hazard penalty.

Deliverable 5: Two-Way Superscalar Processor Design

Purpose

The scalar pipeline is redesigned to fetch and dispatch up to two instructions per cycle when dependencies and resource constraints permit.

Design / Pseudocode



Sample Input

Instruction pair: ADD R3,R1,R2 and LOAD R5,0(R6), with independent operands and separate execution resources.

Sample Output

2-WAY DISPATCH
 Slot 0 -> Integer ALU / Branch execution unit
 Slot 1 -> Load-Store execution unit

Cycle dispatch capacity: up to 2 instructions
 Issue policy: operands ready + execution unit available

Commit policy: in program order using Reorder Buffer

Result / Observation

The design exploits instruction-level parallelism by allowing independent arithmetic and memory instructions to execute in parallel.

Deliverable 6: Out-of-Order and Speculative Execution with Recovery

Purpose

This deliverable explains how the superscalar processor executes ready instructions before older stalled instructions while preserving precise architectural state.

Design / Pseudocode

ON DISPATCH

- allocate physical destination register
- create reorder-buffer entry
- place instruction in reservation station

WHEN operands are ready AND unit is free

- issue instruction (possibly out of program order)

FOR branch

- predict direction and continue speculative fetch

ON branch resolution

IF prediction correct

- continue normally

ELSE

- squash younger speculative instructions
- restore rename/checkpoint state
- redirect PC to correct target
- restart fetch

COMMIT completed instructions in original program order

Sample Input

A long-latency LOAD is followed by an independent ADD and then a predicted branch.

Sample Output

OUT-OF-ORDER EXAMPLE

LOAD R1,0(R2) -> waiting for memory

ADD R5,R6,R7 -> executes early because operands are ready

BRANCH -> predicted and executed speculatively

MISPREDICTION RECOVERY

1. Stop committing wrong-path younger instructions
2. Flush speculative pipeline/queue entries
3. Restore rename-map checkpoint
4. Set PC to correct branch target
5. Resume instruction fetch

Result / Observation

Out-of-order execution improves resource utilization, while the reorder buffer and in-order commit maintain precise exceptions and correct architectural state.

Deliverable 7: Multi-Level Cache and Virtual Memory Design

Purpose

A three-level hierarchy is designed to reduce average access latency while providing sufficient capacity for an edge-computing workload.

Design / Pseudocode

Level	Size	Associativity	Hit Time	Replacement Policy	Technology
L1 I-cache	32 KB	4-way	1 ns	Pseudo-LRU	SRAM
L1 D-cache	32 KB	4-way	1 ns	Pseudo-LRU	SRAM
L2 Unified	512 KB	8-way	5 ns	Pseudo-LRU	SRAM
L3 Shared/Unified	8 MB	16-way	20 ns	SRAM + capacity-oriented design	SRAM
Main Memory	8 GB	—	100 ns	—	DRAM

Virtual memory uses 4 KB pages, a TLB for fast translation, and demand paging. The operating system maps frequently used working-set pages into physical memory while page replacement may use an approximation of LRU such as Clock.

Sample Input

Workload assumptions: instruction working set fits largely in L2/L3; data exhibits temporal locality; occasional TLB misses require page-table access.

Sample Output

CPU -> L1 -> L2 -> L3 -> DRAM

|

v

TLB / Page Table

Virtual address -> TLB translation -> Physical cache/memory access

Chosen technology:

SRAM for L1/L2/L3 because of low latency.

DRAM for main memory because of high density and low cost.

MRAM is considered as a future non-volatile alternative for selected cache/storage levels, but SRAM remains the preferred choice for the fastest cache levels.

Result / Observation

The hierarchy balances speed and capacity: small SRAM L1 caches minimize critical latency, larger L2/L3 caches reduce expensive DRAM accesses, and paging provides a large protected virtual address space.

Deliverable 8: AMAT Calculation and Memory Technology Justification

Purpose

The purpose is to compute Average Memory Access Time (AMAT) for the designed hierarchy and compare it with a simpler two-level alternative.

Design / Pseudocode

$$\begin{aligned} \text{AMAT(3-level)} = & \\ & \text{L1 hit time} \\ & + \text{L1 miss rate} \times (\text{L2 hit time} \\ & + \text{L2 local miss rate} \times (\text{L3 hit time} \\ & + \text{L3 local miss rate} \times \text{DRAM penalty})) \end{aligned}$$

Sample Input

Assumptions: L1 hit time=1 ns, L1 miss rate=5%; L2 hit time=5 ns, L2 local miss rate=10%; L3 hit time=20 ns, L3 local miss rate=20%; DRAM penalty=100 ns.

Sample Output

3-LEVEL CACHE AMAT

$$\begin{aligned} \text{AMAT} &= 1 + 0.05 \times [5 + 0.10 \times (20 + 0.20 \times 100)] \\ &= 1 + 0.05 \times [5 + 0.10 \times 40] \\ &= 1 + 0.05 \times 9 \\ &= 1.45 \text{ ns} \end{aligned}$$

2-LEVEL ALTERNATIVE (assume L2 local miss rate = 10%)

$$\begin{aligned} \text{AMAT} &= 1 + 0.05 \times (5 + 0.10 \times 100) \\ &= 1 + 0.05 \times 15 \\ &= 1.75 \text{ ns} \end{aligned}$$

Improvement with L3 = 1.75 - 1.45 = 0.30 ns (~17.1% lower AMAT)

Result / Observation

The third cache level lowers the average memory access time for the assumed workload. SRAM is justified for latency-critical caches, whereas DRAM remains appropriate for high-capacity main memory.

Deliverable 9: Bandwidth and Throughput Comparison

Purpose

This deliverable estimates throughput and compares scalar versus two-way superscalar execution, as well as two-level versus three-level cache performance.

Design / Pseudocode

THROUGHPUT (IPC) $\approx$ issue width / effective CPI

Instructions per second = clock frequency / CPI
(for scalar CPI model)

For 2-way superscalar:
effective IPC is limited by dependencies, cache misses and branch prediction.

Bandwidth = transfers per second $\times$ bytes per transfer

Sample Input

Assume a 2 GHz clock. Scalar mitigated CPI=1.14. Superscalar effective IPC=1.60 due to dependencies and resource limits. Cache line size=64 bytes; L2 interface supports one 64-byte transfer per cycle.

Sample Output

Alternative	Performance Estimate	Interpretation
Scalar 5-stage	2 GHz / 1.14 = 1.75 GIPS	Baseline throughput
2-way superscalar	2 GHz $\times$ 1.60 IPC = 3.20 GIPS	~83% higher than scalar estimate
2-level cache	AMAT = 1.75 ns	Simpler but more DRAM exposure
3-level cache	AMAT = 1.45 ns	~17.1% lower AMAT
L2 peak bandwidth	2e9 cycles/s $\times$ 64 B = 128 GB/s	Theoretical interface peak

Result / Observation

The two-way superscalar core offers substantially higher potential instruction throughput, while the three-level cache hierarchy improves memory efficiency. Actual gains depend on instruction-level parallelism, miss rates and branch behavior.

Deliverable 10: Reflection, SDG Relevance and Viva Readiness

Purpose

The reflection summarizes the major design trade-offs, links the processor design to SDG 9, and records the learning outcomes required for an oral defense.

Design / Pseudocode

REFLECT ON

- multiple-bus datapath decisions
- pipeline versus superscalar complexity
- forwarding, stalling and prediction trade-offs
- cache capacity versus latency
- SRAM versus DRAM/MRAM choices

MAP TO SDG 9

- resilient digital infrastructure
- innovation in efficient computing hardware
- scalable edge-computing capability

PREPARE VIVA ANSWERS

- Why forwarding?
- Why reorder buffer?
- Why L3 cache?
- Why SRAM for L1?
- How is AMAT computed?

Sample Output

Design decisions: The multiple-bus organization was selected to reduce operand-transfer conflicts. The five-stage pipeline provides a clear and efficient baseline, while the two-way superscalar extension increases throughput at the cost of greater hardware complexity. Forwarding and branch prediction improve CPI, whereas the L1/L2/L3 hierarchy balances latency and capacity.

SDG 9 relevance: Efficient processors and memory systems support innovation, reliable digital infrastructure and scalable edge-computing products. Lower latency and improved performance can help organizations deploy more capable computing systems without simply increasing hardware resources.

Key learning outcomes: register-transfer modeling, datapath design, pipeline hazard analysis, CPI calculation, instruction-level parallelism, out-of-order execution, speculative recovery, cache design, virtual memory, AMAT and performance comparison.

Result / Observation

The complete design study connects processor architecture concepts to measurable performance trade-offs and demonstrates the ability to justify engineering decisions during a viva.

Overall Conclusion

The analytical processor design satisfies the assignment requirements by progressing from a register-transfer/multiple-bus datapath to a five-stage pipeline and then to a two-way superscalar architecture. The study identifies data and control hazards, applies forwarding, stalling and branch prediction, and demonstrates a reduction in CPI from 1.85 to 1.14 under the stated assumptions.

The proposed L1/L2/L3 hierarchy integrates virtual memory and demonstrates an AMAT of 1.45 ns compared with 1.75 ns for the stated two-level alternative. The superscalar design estimates higher instruction throughput, while the cache hierarchy reduces average memory latency. Together, these results provide a complete design and performance analysis suitable for the CSA1221 Computer Architecture assignment.

Appendix: Short Viva Questions and Answers

Viva Question	Short Answer
1. Why is a multiple-bus organization used?	It allows simultaneous operand transfers and reduces bus conflicts.
2. What is a RAW hazard?	A Read After Write dependency where an instruction needs a value produced by an earlier instruction.
3. Why is forwarding useful?	It sends a produced result directly to a dependent stage without waiting for normal write-back.
4. What is speculative execution?	Executing instructions based on a prediction before the prediction is confirmed.
5. How does a reorder buffer help?	It allows out-of-order execution while committing results in program order.
6. Why is SRAM used for L1?	SRAM provides very low latency and does not require refresh.
7. What is AMAT?	Average Memory Access Time, combining hit time and expected miss penalties.
8. Why add L3 cache?	It can reduce costly main-memory accesses and lower AMAT for suitable workloads.